

1 - Models, Eloquent ORM, Factories & Seeders

We now have our database setup and connected to our application. Now we need to start interacting with it. Remember, in MVC, the model, the M deals with the database. We've already been working with routes, controllers and views, now it's time for models. Laravel comes with a great ORM called Eloquent, which has all kinds of methods to work with the data. Methods like find, create, paginate, update, delete and many more. So we don't really create model methods. We already have them available to us. We can also have relationships between models. We're going to create a relationship between users and job listings.

We're going to setup to fetch job listings and we're going to add some listings using Tinker, which is a command line tool where you can essentially run any code you would in your controller. We'll look at forms and input validation within controllers and then we're going to do a bunch of stuff with factories and seeders. This allows us to fill our database with sample data in a single command. This section will help you get to the next level in Laravel development.

2 - Intro To Models

So we know all about routes, controllers, and views. We also learned how to integrate a database and use migrations, but what about models? What are they and how do they fit into the MVC pattern?

Models are the M in MVC. They are responsible for interacting with the database. They represent the data in your application and are used to perform database operations like creating, reading, updating, and deleting records. In Laravel, models are typically stored in the `app/Models` directory.

Eloquent ORM

Laravel uses Eloquent ORM (Object-Relational Mapping) to interact with the database. Eloquent is an implementation of the Active Record pattern. It allows you to work with database records as objects. This makes it easy to query the database and perform CRUD operations. This allows us to keep our models clean and simple. There are tons of pre-built methods that we can use to interact with the database.

Create A Basic Model

Before we get into Eloquent ORM, let's create a basic model. We can use Artisan to create models, but we will create this one manually. Create a new file called `Job.php` in the `app/Models` directory. Add the following code:

```
<?php
namespace App\Models;

class Job
{
    static public function all(): array
    {
        return [
            [
```

```
        "id" => 1,
        "title" => "Software Engineer",
        "description" => "Design and develop high-quality software applications,
collaborating with teams and ensuring efficient solutions.",
    ],
    [
        "id" => 2,
        "title" => "Marketing Specialist",
        "description" => "Develop and execute marketing campaigns, conduct market
research, and drive brand engagement.",
    ],
    [
        "id" => 3,
        "title" => "Customer Support Representative",
        "description" => "Provide excellent customer service, troubleshoot
customer issues, and maintain customer satisfaction.",
    ],
];
}
```

The namespace `App\Models` tells Laravel where to find the model. There could be other classes with the same name of `Job` in different namespaces. Namespaces help us categorize our classes and avoid conflicts.

We are creating a static class. This class has a static method called `all` that returns an array of job listings. It is also typed to an array type, which is optional.

This is a simple example to demonstrate how models work. Usually the data would come from a database, but we are just hardcoding it for now.

Use The Model

Now that we have a model, let's use it in our controller. Open the `JobController.php` file and import the `Job` model at the top:

```
use App\Models\Job;
```

Now replace the `index` method with the following code:

```
public function index(): View
{
    $title = 'Available Jobs';
    $jobs = Job::all();
    return view('jobs/index', compact('title', 'jobs'));
}
```

We are now passing the jobs to the view. We can access the jobs in the view using the `$jobs` variable. You'll get an error now because before we were passing an array of strings, but now we are passing an associative array. We need to update the view to handle this new data structure.

Open the `jobs/index.blade.php` file and replace the `foreach` loop with the following code:

```
@foreach($jobs as $job)
    <li>{{ $job['title'] }}</li>
@endforeach
```

Now we are looping through the jobs and displaying the title. You can also display the description with `$job['description']`, but we are not keeping this code. In fact you can delete the `app/models/Job.php` file because we will generate a new one in the next video.

In the next lesson, we will learn how to use Eloquent ORM to interact with the database.

3 - Fetching Data & Eloquent

As I mentioned in the last video, Eloquent is an ORM or Object Relational Mapper, which is a powerful tool that allows us to interact with the database. In this lesson, we will learn how to use Eloquent to fetch data from the database.

Generating a Model

Let's re-create the Job model. We can actually generate a new model using the `make:model` command. This command will create a new model in the `app/Models` folder.

```
php artisan make:model Job
```

This will create a file at `app/Models/Job.php` that looks like this:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Job extends Model
{
    use HasFactory;
}
```

This is a basic model that extends the `Model` class. The `Model` class is provided by Eloquent and it has a lot of useful methods that we can use to interact with the database. The `HasFactory` trait is used to generate factories for the model. We will learn more about factories in a later lesson.

Create Model & Migration

I also want to mention that you can create a model and a migration at the same time using the `make:model` command. This is useful if you want to create a model and a migration for a new table. You can do this by passing the `--migration` flag to the `make:model` command.

You don't have to do this because we already created the migration, but I wanted to show you how to do it.

```
php artisan make:model Job --migration
```

This would create a model at `app/Models/Job.php` and a migration at `database/migrations/2022_01_01_000000_create_jobs_table.php`.

Specifying the Table Name

By default, it will look for a table with the same name as the model, but with the `s` suffix. In this case, it will look for a table called `jobs`. We don't want that since our table is called `job_listings`. We can specify the table name by adding a protected property to the model:

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Job extends Model
{
    protected $table = 'job_listings';
}
```

Now we can use this model to fetch data from the database.

Mass Assignment

Eloquent provides a way to protect against mass assignment vulnerabilities. This is a security feature that prevents users from modifying columns that they shouldn't be able to modify. To use this feature, we need to specify the columns that we want to allow mass assignment for. Mass assignment is when we pass an array of data to a model and the model will automatically assign the data to the columns in the database. We want to be able to mass assign most fields, for now, we just need to add the title and description fields.

```
class Job extends Model
{
    use HasFactory;

    protected $table = 'job_listings';
```

```
}
```

Fetching Data

The code that we have in the controller should still work because we are still using the `Job` model and the `all` method. The difference is that now we are fetching data from the database instead of a hardcoded array.

Arrow Syntax

In your `resources/views/jobs/index.blade.php` file, you can update the code to use the arrow syntax to access the properties of the job object. \

```
<x-layout>
  <ul>
    @forelse($jobs as $job)
      <li>{{ $job->title }}</li>
    @empty
      <li>No jobs found</li>
    @endforelse
  </ul>
</x-layout>
```

I am also going to just use `with()` now that we are only passing in a single variable. This is a little cleaner.

```
public function index(): View
{
    $jobs = Job::all();
    return view('jobs/index')->with('jobs', $jobs);
}
```

There is no data in the database yet. Let's add some. You could do this with PG Admin or the shell. I am going to use Tinker. We will do this in the next lesson.

4 - Tinker & CRUD Operations

In the last lesson, we created a model and used it to fetch data from the database. In this lesson, I want to show you how to create, read update and delete data using Eloquent. We will be using Tinker, which is a command line tool, however, when we want to do this stuff in our application, we use the same code.

Let's run Tinker:

```
php artisan tinker
```

Access Models & Data

We can access the Job model by using the fully qualified namespace:

```
>>> App\Models\Job::all()
```

This will get our listings, which right now we do not have any.

We can see the schema of the model by using the `getColumnListing` method on the `Schema` facade:

```
Schema::getColumnListing('job_listings');
```

One thing I want to do is create a variable for the model namespace. This will make it easier to type and will help us avoid errors.

```
>>> $job = App\Models\Job::class
```

Now you should be able to do the following:

```
>>> $job::all()
```

And see the same result.

Adding Data

Let's add some data to the database. We can do this by creating a new instance of the model and setting the properties:

```
$Job::create([
    'title' => 'Job One',
    'description' => 'This is an description for job one',
]);
```

You should see a result like this:

```
App\Models\Job {#5996
  title: "Job One",
  description: "This is an description for job one",
  updated_at: "2024-08-13 11:43:25",
  created_at: "2024-08-13 11:43:25",
}
```

```
    id: 1,  
  }
```

Let's add few more:

```
$job::create([  
  'title' => 'Job Two',  
  'description' => 'This is an description for job two',  
]);
```

```
$job::create([  
  'title' => 'Job Three',  
  'description' => 'This is an description for job three',  
]);
```

```
$job::create([  
  'title' => 'Job Four',  
  'description' => 'This is an description for job four',  
]);
```

Now list the jobs:

```
>>> $job::all()
```

You should see all jobs listed.

Finding Data

We can use the `find` method to find a specific job. We just need to pass in the ID.

Let's find the first job:

```
>>> $job::find(1)
```

You should see the first job.

Updating Data

Let's update the first job:

```
>>> $job::find(1)->update(['title' => 'Updated Job One'])
```

Now check it:

```
>>> $job::find(1)
```

You should see the updated job.

Deleting Data

Let's delete the second job:

```
>>> $job::find(2)->delete()
```

Now check the jobs:

```
>>> $job::all()
```

You should see the second job is gone.

So it's as easy as that to perform CRUD operations with Tinker. If you go to the jobs page, you should see the jobs listed there as well.

5 - Model Binding & Single Job Listing

We have the titles of the jobs on the /jobs page. Let's add links that will take them to the single listing page.

We already have a `show` method in the `JobController`. This is the method we will use to show a single listing. Right now, it is just returning a string. Let's create a view for it.

Create a file at `resources/views/jobs/show.blade.php` and add the following code:

```
<x-layout>
  <h1 class='text-2xl'>{{ $job->title }}</h1>
  <p>{{ $job->description }}</p>
</x-layout>
```

Now, let's update the `show` method in the `JobController` to return this view:

```
public function show(Job $job): View
{
```



```
    return view('jobs.show', compact('job'));  
}
```

Model Binding

We are using a feature called model binding. This is a feature of Laravel that allows us to type-hint a model in a controller method and Laravel will automatically fetch the model from the database based on the route parameter. In this case, the route parameter is `job`.

Then we are just loading the view and passing the job to it. You can pass as an array or use the compact helper function.

Now if you go to `http://127.0.0.1:8000/jobs/1` you should see the job title and description.

Let's add the link to the title.

route helper

The `route` helper is a function that allows us to generate a URL to a named route. In this case, we are using the `jobs.show` route.

Open the `resources/views/jobs/index.blade.php` file and add the following around the job title:

```
<a href="{{ route('jobs.show', $job->id) }}">  
    {{ $job->title }}  
</a>
```

Now if you go to the `/jobs` page, you should see the job titles as links. Clicking on them will take you to the single job listing page.

Remember all of this will look much nicer and we will have a lot more fields. I just want to keep the data and views simple for now so you understand what's going on.

6 - Create a New Job

So we know how to fetch jobs within our application. We also know how to create them from within Tinker. But how do we actually create them from within our application? It actually isn't much different than what we did in Tinker. We already have a form on the create page if you have been following along. It is extremely ugly at the moment but that's okay. Remember, I am not interested in making things look good yet or having all of the data for the job listings. We are only dealing with the title and description at the moment for simplicity.

The form is being loaded from the Job Controller's create method, which loads the view at `resources/views/jobs/create.blade.php`.

The form is being submitted to the store method on the Job Controller, which right now looks like this:

```
public function store(Request $request): string
{
    $title = $request->input('title');
    $description = $request->input('description');

    return "Title: $title, Description: $description";
}
```

So we already know how to get the data using the Request object. We also know how to create a new job using the Eloquent ORM from what we did with Tinker. Let's apply that knowledge to our application.

First, let's update the store method to create a new job:

```
public function store(Request $request): RedirectResponse
{
    $title = $request->input('title');
    $description = $request->input('description');

    Job::create([
        'title' => $title,
        'description' => $description
    ]);

    return redirect()->route('jobs.index');
}
```

Since we are using the `RedirectResponse` class, we need to import it at the top of the file:

```
use Illuminate\Http\RedirectResponse;
```

Now when we submit the form, it will create a new job and redirect us to the jobs index page. We can see the job that we just created.

Now that we are able to fetch and create jobs, I think the next step is to add the rest of the fields to the database table and then work on the styling of the application.

7 - Input Validation & Errors

Right now we can create a new job but if we try and submit an empty form it will break because the title field can not be null. We need to add some validation to the form. We will use Laravel's built-in validation rules.

We have access to the request object in the controller. We can use the `validate` method to validate the request. The `validate` method will throw an exception if the validation fails. We can use the `withErrors` method to add the errors to the session. We can then use the `old` method to display the errors in the view.

Let's add some validation to the `store` method in the `JobListingController`. Add the following code to the `store` method:

```
public function store(Request $request): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
    ]);

    // Create a new job listing with the validated data
    Job::create([
        'title' => $validatedData['title'],
        'description' => $validatedData['description'],
    ]);

    return redirect()->route('jobs.index');
}
```

In this code, we are using the `validate` method to validate the incoming request data. We are passing an array of validation rules to the `validate` method. The `validate` method will return the validated data if the validation passes. If the validation fails, it will throw an exception.

We are using the `required` rule to make sure the title and description fields are not empty. We are using the `string` rule to make sure the title is a string. We are using the `max` rule to make sure the title is no longer than 255 characters. You can find all of the validation rules [here](#).

If the validation passes, we are creating a new job listing with the validated data. If the validation fails, the user will be redirected back to the form with the errors.

Displaying Errors With `@error`

We can display the errors with the `@error` directive. Add the following code to the `/resources/views/jobs/create.blade.php` file:

```
<form action="/jobs" method="POST">
    @csrf
    <input type="text" name="title" placeholder="Title">
    <!-- Error Message for Title -->
    @error('title')
    <div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
    @enderror
    <input type="text" name="description" placeholder="Description">
    <!-- Error Message for Description -->
    @error('description')
    <div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
    @enderror
</form>
```

```
<button type="submit">Submit</button>
</form>
```

Again, this is very ugly. We will fix the styling later after we add the rest of the fields.

Now if you submit the empty form you will see the errors. We still have an issue though. Try and just submit the title and leave the description blank. You see the error but the value that you typed goes away. We need to add the `old` method to the form values.

old Method

The `old` method is used to repopulate the form fields with the old input. This is useful when the form fails validation and you want to repopulate the form with the old input. Add the following code to the `/resources/views/jobs/create.blade.php` file:

```
<form action="/jobs" method="POST">
  @csrf
  <input type="text" name="title" placeholder="Title" value="{{ old('title')
  }}">
  <!-- Error Message for Title -->
  @error('title')
  <div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
  @enderror
  <input type="text" name="description" placeholder="Description" value="{{
  old('description') }}">
  <!-- Error Message for Description -->
  @error('description')
  <div class="text-red-500 mt-2 text-sm">{{ $message }}</div>
  @enderror
  <button type="submit">Submit</button>
</form>
```

Now only fill in the title and click submit. You will see the title is repopulated with the old input.

8 - Update Job Schema & Migration

So we have very basic create and read functionality for jobs. The next step is to update the job schema and migration to include the rest of the fields that we need for a job listing. We will also update the form to include these fields. A schema is just a definition of the table and the fields that it contains. A migration is a way to update the schema of a table.

Create a New Migration

When we want to change the schema of a table we need to create a new migration. Technically you could do a rollback and then use the same migration file, but usually that would be a bad idea because it would be hard to track down the changes that were made. ANY changes that you make should be in a new migration file.

An issue that we could run into when we run a migration that adds fields to the table is getting an error because there is already data in the table. We have a bunch of options here. You could manually delete the records first, but another approach is to clear the data in the migration file. This is what we will do.

We need to run the following command to create a new migration:

```
php artisan make:migration add_fields_to_job_listings_table --table=job_listings
```

We need to specify the table name because we didn't use the regular convention of using the plural form of the model name because there was already a table named `jobs` in the database. We used the `--table` option to specify the table name.

Here is an example of a job listing and the fields that we need:

```
[
  "id" => 1,
  "user_id" => 1,
  "title" => "Software Engineer",
  "description" => "As a Software Engineer at Algorix, you will be responsible
for designing, developing, and maintaining high-quality software applications. You
will work closely with cross-functional teams to deliver scalable and efficient
solutions that meet business needs. The role involves writing clean, maintainable
code, participating in code reviews, and staying current with industry trends to
ensure our technology stack remains cutting-edge.",
  "salary" => 90000,
  "tags" => "development, coding, java, python",
  "job_type" => "Full-time",
  "remote" => false,
  "requirements" => "Bachelors degree in Computer Science or related field, 3+
years of software development experience",
  "benefits" => "Healthcare, 401(k) matching, flexible work hours",
  "address" => "123 Main St",
  "city" => "Albany",
  "state" => "NY",
  "zipcode" => "12201",
  "contact_email" => "info@algorix.com",
  "contact_phone" => "348-334-3949",
  "company_name" => "Algorix",
  "company_description" => "Algorix is a leading tech firm specializing in
innovative software solutions and cutting-edge technology.",
  "company_logo" => "logos/logo-algorix.png",
  "company_website" => "https://algorix.com"
];
```

up Method

Open the new migration file and add the following code to the `up` method to add the new fields to the table:

```

public function up(): void
{
    // Clear the table
    DB::table('job_listings')->truncate();

    // Modify the table schema
    Schema::table('job_listings', function (Blueprint $table) {
        $table->integer('salary');
        $table->string('tags')->nullable();
        $table->enum('job_type', ['Full-Time', 'Part-Time', 'Contract', 'Temporary',
'Internship', 'Volunteer', 'On-Call'])->default('Full-Time');
        $table->boolean('remote')->default(false);
        $table->text('requirements')->nullable();
        $table->text('benefits')->nullable();
        $table->string('address')->nullable();
        $table->string('city');
        $table->string('state');
        $table->string('zipcode')->nullable();
        $table->string('contact_email');
        $table->string('contact_phone')->nullable();
        $table->string('company_name');
        $table->text('company_description')->nullable();
        $table->string('company_logo')->nullable();
        $table->string('company_website')->nullable();
    });
}

```

We are first truncating the table to clear the data. This is not something that you would normally do in a migration, but we are doing it here because we are adding new fields to the table and we don't want to get an error because there is already data in the table. Truncate will not only delete the records, but it will also reset the auto-incrementing id.

We are adding a bunch of new fields. It is all pretty self-explanatory. For job type, we are using an enum to limit the values to a set of predefined values and using a default of **Full-Time**. For remote, we are using a boolean to indicate if the job is remote or not. For the rest of the fields, we are using the regular string and text types. The tags will be a comma-separated list of tags. Anything labeled as **nullable** can be left blank.

down Method

For the down, we will just drop all of the columns we added in the **up** method. Add the following code to the **down** method:

```

public function down(): void
{
    Schema::table('job_listings', function (Blueprint $table) {
        $table->dropColumn(['salary', 'tags', 'job_type', 'remote',
'requirements', 'benefits', 'address', 'city', 'state', 'zipcode',
'contact_email', 'contact_phone', 'company_name', 'company_description',
'company_logo', 'company_website']);
    });
}

```

```
});  
}
```

Don't run the migration just yet because I want to add a `user_id` field but I want to talk a little bit about relationships first and we'll do that in the next lesson.

Mass Assignment

We need to update the `JobListing` model to include the new fields. We also need to update the `$fillable` property to include the new fields. Here is the updated model:

```
class JobListing extends Model  
{  
    use HasFactory;  
  
    protected $fillable = [  
        'title',  
        'description',  
        'salary',  
        'tags',  
        'job_type',  
        'remote',  
        'requirements',  
        'benefits',  
        'address',  
        'city',  
        'state',  
        'zipcode',  
        'contact_email',  
        'contact_phone',  
        'company_name',  
        'company_description',  
        'company_logo',  
        'company_website'  
    ];  
}
```

In the next lesson, we will talk about relationships in Eloquent.

9 - Eloquent Relationships

In this lesson, we will learn about Eloquent relationships which allow you to define relationships between models. This is a powerful feature of Eloquent that allows you to easily work with related data. In our app, we want to be able to associate a job listing with a user.

Types of Relationships

There are several types of relationships that you can define in Eloquent. These relationships do not apply only to Eloquent or Laravel or PHP. They are part of the database design and are used to model the relationships between tables in a database. The most common types of relationships are:

- One-to-One: A one-to-one relationship is a relationship between two tables where each record in one table is related to exactly one record in the other table. An example would be user profiles. A single user can have a single profile.
- One-to-Many: A one-to-many relationship is a relationship between two tables where each record in one table is related to zero, one, or many records in the other table. An example would be blog posts. A single user can write multiple blog posts, but each blog post is written by a single user.
- Many-to-Many: A many-to-many relationship is a relationship between two tables where each record in one table is related to zero, one, or many records in the other table, and vice versa. An example would be students and courses. A student can enroll in many courses, and each course can have many students.

Defining Relationships In Models

In our case, we want to define a one-to-many relationship between the `JobListing` model and the `User` model. This means that each `JobListing` can only have one `User` and each `User` can have many `JobListings`.

What this will allow us to do within our application is to easily access the user that created a job listing. We can also easily access all of the job listings that a user has created.

For instance:

```
$user = User::find(1);  
$jobListings = $user->jobListings;
```

or

```
$jobListing = JobListing::find(1);  
$user = $jobListing->user;
```

To define this relationship, we need to add a method to the `JobListing` model that defines the relationship. Add the following method to the `JobListing` model:

```
public function user(): BelongsTo  
{  
    return $this->belongsTo(User::class);  
}
```

Import the `BelongsTo` class at the top of the file:


```
use Illuminate\Database\Eloquent\Relations\BelongsTo;
```

This method defines a `belongsTo` relationship between the `JobListing` model and the `User` model. The `belongsTo` method tells Eloquent that the `JobListing` model belongs to the `User` model. This means that each `JobListing` record will have a `user_id` column that references the `id` column of the `User` model.

Next, we need to define the inverse of this relationship in the `User` model. Add the following method to the `User` model:

```
public function jobListings(): HasMany
{
    return $this->hasMany(JobListing::class);
}
```

Import the `HasMany` class at the top of the file:

```
use Illuminate\Database\Eloquent\Relations\HasMany;
```

This method defines a `hasMany` relationship between the `User` model and the `JobListing` model. The `hasMany` method tells Eloquent that the `User` model has many `JobListing` records. This means that each `User` record can have many `JobListings` associated with it.

Update Migration

Now that we have defined the relationship between the `JobListing` and `User` models, we need to update the `job_listings` table migration to add a `user_id` column. Add the following code to the `up` method of the migration:

```
Schema::table('job_listings', function (Blueprint $table) {
    // Add this line
    $table->unsignedBigInteger('user_id')->after('id');
    //...
```

We are specifying a new field called `user_id` that is an unsigned big integer which is a positive number with a wide range. We are also specifying that it should be added after the `id` column.

Next, we need to add a foreign key constraint to the `user_id` column. Add the following code at the bottom of the `up` method:

```
Schema::table('job_listings', function (Blueprint $table) {
    // ...

    // Adding a foreign key constraint
```

```
$table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');
});
```

This code adds a foreign key constraint to the `user_id` column. The `references` method specifies that the `user_id` column references the `id` column of the `users` table. The `onDelete('cascade')` method specifies that if a `User` record is deleted, all of its associated `JobListing` records should also be deleted.

Finally, we need to update the `down` method of the migration to remove the `user_id` column and the foreign key constraint. Add the following code to the `down` method:

```
public function down(): void
{
    Schema::table('job_listings', function (Blueprint $table) {
        // Drop foreign key constraint and user_id column
        $table->dropForeign(['user_id']);
        $table->dropColumn('user_id');

        // ...
    });
}
```

One last thing you need to do is add the `user_id` to the `$fillable` property of the `JobListing` model. Add the following code to the `JobListing` model:

```
protected $fillable = [
    'user_id',
    // ...
];
```

Now we are ready to run our migration. Run the following command:

```
php artisan migrate
```

Now you should have all of those fields in your `job_listing` table. Now that our table is all set, I want to start to seed some data using factories.

10 - Using Factories

In this lesson, we will learn how to use factories. Factories are a way to define the attributes of a model in a reusable way. They allow you to define a blueprint for creating model instances with predefined attributes. This is useful when you need to seed your database with sample data or when you need to create model instances in your tests.

Laravel provides a factory class that you can use to define factories for your models. Factories are typically stored in the `database/factories` directory of your Laravel application.

If we look in the `database/factories` directory, we will see a file called `UserFactory.php`.

As you can see, in their most basic form, factories are classes that extend Laravel's base `Factory` class and define a `definition` method that returns the default set of attribute values that should be applied when creating a model using the factory.

Here is what the `definition` method looks like in the `UserFactory.php` file:

```
public function definition(): array
{
    return [
        'name' => fake()->name(),
        'email' => fake()->unique()->safeEmail(),
        'email_verified_at' => now(),
        'password' => static::$password ??= Hash::make('password'),
        'remember_token' => Str::random(10),
    ];
}
```

Laravel comes bundles with a library called Faker that allows you to generate fake data. This is useful when you need to seed your database with sample data or when you need to create model instances in your tests.

In this case, it will generate the following:

- **name:** A random name
- **email:** A random email address that is unique
- **email_verified_at:** A timestamp of right now
- **password:** A hashed password of the string `password`
- **remember_token:** A random string of 10 characters

Running Factories

We can create something called a seeder that will run factories, but we can also run them within our application, in tests as well as within Tinker. Let's use the user factory to create some users. Open up Tinker by running `php artisan tinker` and run the following command:

```
\App\Models\User::factory()->create();
```

This will create a single user. Let's create 10 more by using the `count` method:

```
\App\Models\User::factory()->count(10)->create();
```

Now you should have 11 users in your database. You can check with the following command:

```
\App\Models\User::all();
```

As you can see these are verified users. There is a date for the field `email_verified_at`. However, if you wanted a user that was not verified, you can use the `unverified` method:

```
\App\Models\User::factory()->unverified()->create();
```

This user will have a null value for the `email_verified_at` field.

Now that you know what a factory is and how to use it, let's create a factory for our `JobListing` model in the next lesson.

11 - Creating Factories

Now that we saw how the user factory works, let's create a factory for the `Job` model.

Creating a Factory

To create a factory for the `Job` model, run the following command:

```
php artisan make:factory JobFactory
```

This will create a file at `database/factories/JobFactory.php`. It will create a class called `JobFactory` that extends the `Factory` class with a method called `definition`. The `definition` method returns an array of attributes that will be used to create a new `Job` model. Same as with the user factory that we looked at in the last lesson.

We need to have a user for each listing, so let's bring in the `User` model:

```
use App\Models\User;
```

Now, let's define the `definition` method:

```
public function definition(): array
{
    return [
        'user_id' => User::factory(), // Create new user for each listing
        'title' => $this->faker->jobTitle,
        'description' => $this->faker->paragraphs(3, true),
        'salary' => $this->faker->numberBetween(40000, 120000),
        'tags' => implode(', ', $this->faker->words(3)),
        'job_type' => $this->faker->randomElement(['Full-Time', 'Part-Time',
```

```

    'Contract', 'Temporary', 'Internship', 'Volunteer', 'On-Call']],
    'remote' => $this->faker->boolean,
    'requirements' => $this->faker->sentences(3, true),
    'benefits' => $this->faker->sentences(2, true),
    'address' => $this->faker->streetAddress,
    'city' => $this->faker->city,
    'state' => $this->faker->state,
    'zipcode' => $this->faker->postcode,
    'contact_email' => $this->faker->safeEmail,
    'contact_phone' => $this->faker->phoneNumber,
    'company_name' => $this->faker->company,
    'company_description' => $this->faker->paragraphs(2, true),
    'company_logo' => $this->faker->imageUrl(100, 100, 'business', true,
'logo'),
    'company_website' => $this->faker->url,
];
}

```

Here is an explanation of each attribute:

- **user_id**: Creates a new user for each job listing using the - User::factory() method.
- **title**: Generates a random job title.
- **description**: Creates a description with three paragraphs.
- **salary**: Random salary between 40,000 and 120,000.
- **tags**: Generates a string of three random words separated by commas.
- **job_type**: Selects a random job type from the predefined list.
- **remote**: Randomly determines if the job is remote or not.
- **requirements**: Generates a string of three sentences for job requirements.
- **benefits**: Generates a string of two sentences for job benefits.
- **address**: Generates a random street address.
- **city**: Generates a random city name.
- **state**: Generates a random state name.
- **zipcode**: Generates a random postal code.
- **contact_email**: Generates a safe email address.
- **contact_phone**: Generates a random phone number.
- **company_name**: Generates a company name.
- **company_description**: Creates a company description with two paragraphs.
- **company_logo**: Generates a URL to a random image (representing a company logo).
- **company_website**: Generates a random URL for the company's website.

Now that we have a factory for the **Job** model, let's use it to create some jobs.

Run The Job Factory

To create a job listing using the factory, open Tinker by running **php artisan tinker** and run the following command:

```
\App\Models\Job::factory()->create();
```

Let's create 10 more by using the `count` method:

```
\App\Models\Job::factory()->count(10)->create();
```

Yes sir! You should have 11 jobs in your database. You can check with the following command:

```
\App\Models\Job::all();
```

You can also go to your app and you should see the titles of the jobs on the `/jobs` page.

How cool is that? All this right out of the box. Imagine how much time you would have saved if you had to create all these jobs manually. This is the power of Laravel and its ecosystem.

12 - Seeders

We know how to create and use factories and we used them within Tinker. We can create seeders to populate our database with data right from the command line with Artisan. This is useful for testing and development purposes. We can also use seeders to populate our database with initial data when we deploy our application.

If you open the `database/seeders` directory, you will see a file called `DatabaseSeeder.php`. This is the default seeder that Laravel creates for us. Seeders only have a single method called `run`. This method is called when we run the `db:seed` command. This particular seeder uses the user factory to create a user.

Create a Random User Seeder

Let's create a user seeder that will fun our factory and create 10 random users on the fly.

Create the seeder with the following command:

```
php artisan make:seeder RandomUserSeeder
```

This will create a new file at `database/seeders/RandomUserSeeder.php`. This file will contain a class called `RandomUserSeeder` that extends the `Seeder` class. It will have a method called `run`. This is where we will define the logic for populating our database with data.

Bring in the `User` model:

```
use App\Models\User;
```

Add the following code to the `run` method:

```
public function run(): void
{
    // Create 10 users using the UserFactory
    $users = User::factory(10)->create();

    echo "Users created successfully!";
}
```

We can run the seeder with the following command:

```
```bash
php artisan db:seed --class=RandomUserSeeder
```

## Create a Random Job Seeder

Let's create a new seeder called `RandomJobSeeder.php` in the `database/seeder` directory. We can use the `make:seeder` command to create a new seeder.

```
php artisan make:seeder RandomJobSeeder
```

This will create a file at `database/seeder/RandomJobSeeder.php`. It will create a class called `RandomJobSeeder` that extends the `Seeder` class with a method called `run`. The `run` method is where we will define the logic for populating our database with data.

Let's bring in the `Job` model:

```
use App\Models\Job;
```

Now in the `run` method, we will use the `Job` model to create a new job listing. We will use the `factory` method to create a new job listing. We will use the `count` method to specify the number of job listings we want to create. We will use the `create` method to create the job listing.

```
public function run(): void
{
 // Generate 10 job listings using the factory
 Job::factory()->count(10)->create();

 echo "Jobs created successfully!";
}
```

This will create 10 job listings using the `JobFactory` that we created earlier. We can now run the seeder using the `db:seed` command:

```
php artisan db:seed --class=RandomJobSeeder
```

This will add 10 job listings to our database. With 10 new users.

If you want to keep it this way you can, but I would like to have a seeder that creates the same group of job listings. In the next lesson, we will have the seeder put in some hardcoded jobs and we will look at truncating the tables first and calling seeders from within another using the `call` method.

## 14 - Final Database Seeder

We have a way to seed random jobs and users, which is ok, but I want to work with the same group of jobs. I also want to truncate the tables first. We can do this by using the `truncate` method on the `DB` facade. We can also use the `call` method to call another seeder from within another seeder.

Attached to this lesson is a download for a `job_listings.php` file which returns an array of 10 job listings. I want to use this to seed our database. Put this file in the `database/seeder/data` directory. You will have to create the `data` directory.

Let's create a new seeder called `JobSeeder.php` in the `database/seeder` directory. We can use the `make:seeder` command to create a new seeder.

```
php artisan make:seeder JobSeeder
```

This will create a file at `database/seeder/JobSeeder.php`. This file will contain a class called `JobSeeder` that extends the `Seeder` class. It will have a method called `run`. This is where we will define the logic for populating our database with data.

We need to bring in the `DB` facade and the `User` model:

```
use Illuminate\Support\Facades\DB;
use App\Models\User;
```

This is because we will be using the `DB` facade to insert data and we need to get a user id for each job.

Add the following code for the `run` method:

```
public function run(): void
{
 // Load job listings data
 $jobListings = include database_path('seeder/data/job_listings.php');

 // Get all user IDs
 $userIds = User::pluck('id')->toArray();
```



```
foreach ($jobListings as &$amp;listing) {
 // Assign a random user_id to each job listing
 $listing['user_id'] = $userIds[array_rand($userIds)];
 // Add timestamps
 $listing['created_at'] = now();
 $listing['updated_at'] = now();
}

// Insert job listings
DB::table('job_listings')->insert($jobListings);
}
```

We are getting the data from the `job_listings.php` file and then assigning a random user id to each job listing.

We are using the `pluck` method to get all the user ids and then we are using the `array_rand` function to get a random user id.

We are also adding timestamps to each job listing.

We are then using the `insert` method on the `DB` facade to insert the data into the database.

We can call this seeder on it's own, however what I would like to do is call this along with the user seeder in the main `DatabaseSeeder.php` file. We can do this by using the `call` method. I also want to truncate the tables first. We can do this by using the `truncate` method on the `DB` facade.

Open the `DatabaseSeeder.php` file and add bring in the `DB` facade:

```
use Illuminate\Support\Facades\DB;
```

Now add the following code to the `run` method:

```
public function run(): void
{
 // Truncate tables
 DB::table('job_listings')->truncate();
 DB::table('users')->truncate();

 $this->call(RandomUserSeeder::class);
 $this->call(JobSeeder::class);
}
```

We are clearing the tables and then calling the user seeder, which creates 10 random users. We are then calling the job seeder, which creates our 10 listings from the array and uses a random user id for each listing.

Now run the seeder:

```
php artisan db:seed
```

Now whenever you want to reset the database, you can run the `db:seed` command. This will clear the tables and then add the 10 job listings and 10 random users to our database. Now we have some data to work with. In the next section, we will start to work on the presentation of our site.